INFORME DE ACTIVIDADES
SEMANA 09

Codificación del primer requerimiento en React JS
Ventana de ingreso (Login)

Proyecto: Pequeños Exploradores LMS
Área: Ingeniería de Sistemas y Computación
Tecnologías: React, TypeScript, Vite y CSS

Huancayo — Perú | 2026

1. Introducción

La presente actividad desarrolla el primer requerimiento de interfaz del sistema Pequeños Exploradores LMS: la ventana de ingreso de usuarios. El módulo se ha extraído y adaptado del prototipo de React previamente elaborado para el proyecto, conservando la distinción entre los espacios familiar e institucional y los perfiles administrativo, docente y apoderado.

La entrega se limita a un fragmento ejecutable del front-end, acompañado por sus archivos de configuración y estilos. La validación utiliza registros ficticios en memoria y sirve para demostrar el comportamiento visual y lógico del formulario; no representa un sistema de autenticación conectado a un servidor.

2. Objetivos

Objetivo general. Codificar la interfaz de inicio de sesión de Pequeños Exploradores LMS con React JS, incorporando controles de entrada, selección de modalidad y validación básica de usuarios de demostración.

Objetivos específicos. (a) Construir un formulario reutilizable y adaptable a dispositivos; (b) controlar los datos mediante estados de React; (c) diferenciar el acceso familiar del institucional; (d) mostrar mensajes de error cuando la validación no se cumple; y (e) entregar un fragmento de código independiente y ejecutable.

3. Alcance del primer requerimiento

El requerimiento permite seleccionar el espacio de acceso, elegir el perfil institucional cuando corresponde, escribir usuario y contraseña, ejecutar la validación y mostrar una pantalla de bienvenida cuando la combinación coincide con una cuenta activa. El campo de contraseña utiliza type="password" y el formulario admite envío mediante su botón o la tecla Enter.

3.1. Reglas funcionales

RF-01. El usuario puede alternar entre modalidad familiar e institucional.

RF-02. En la modalidad institucional se selecciona un perfil: administrativo, docente o apoderado.

RF-03. El formulario solicita usuario y contraseña obligatorios.

RF-04. La verificación compara usuario, contraseña, perfil, modalidad y estado activo.

RF-05. Ante datos incorrectos se muestra un mensaje y no se concede acceso.

RF-06. Ante datos correctos se muestra una bienvenida y una opción para cerrar la sesión de demostración.

4. Estructura del fragmento entregado

```
semana09-login/
  index.html
  package.json
  tsconfig.json
  README.md
  src/
```

```
    main.tsx
    App.tsx
    style.css
```

App.tsx reúne la lógica del componente y el formulario; main.tsx monta React; style.css define la apariencia y el comportamiento adaptable; package.json declara las dependencias y comandos de Vite.

5. Desarrollo de la codificación

5.1. Tipos y datos de prueba

```
type Role = 'admin' | 'docente' | 'padre';
type Space = 'familiar' | 'institucional';
type User = {
  username: string; password: string; role: Role;
  space: Space; name: string; status: 'activo' | 'pendiente';
};
```

Los tipos permiten limitar los valores de modalidad y rol. El arreglo users contiene exclusivamente cuentas ficticias de prueba y no sustituye una base de datos.

5.2. Estados controlados con React

```
const [space, setSpace] = useState<Space>('familiar');
const [role, setRole] = useState<Role>('padre');
const [username, setUsername] = useState('');
const [password, setPassword] = useState('');
const [error, setError] = useState('');
const [currentUser, setCurrentUser] = useState<User | null>(null);
```

useState conserva el valor de cada campo y permite actualizar la interfaz cuando el usuario cambia de modalidad, selecciona un rol o intenta ingresar.

5.3. Validación del ingreso

```
const login = (event: FormEvent<HTMLFormElement>) => {
  event.preventDefault();
  const found = users.find(u =>
    u.username.trim().toLowerCase() === username.trim().toLowerCase() &&
    u.password === password && u.role === role &&
    u.space === space && u.status === 'activo'
  );
  if (!found) {
    setError('Credenciales incorrectas, rol distinto o cuenta pendiente.');
    return;
  }
  setError(''); setCurrentUser(found);
};
```

El método find localiza una coincidencia exacta de credenciales y restricciones de acceso. En el proyecto original, la lógica equivalente está integrada en App.tsx y conduce al panel principal; en este fragmento aislado se reemplaza dicho panel por una bienvenida para delimitar el primer requerimiento.

5.4. Formulario y controles

```
<form onSubmit={login}>
  <label>Usuario
    <input required value={username}
      onChange={e => setUsername(e.target.value)} />
  </label>
  <label>Contraseña
    <input required type="password" value={password}
```

```
      onChange={e => setPassword(e.target.value)} />
  </label>
  {error && <p role="alert">{error}</p>}
  <button type="submit">Ingresar a mi espacio</button>
</form>
```

El atributo required evita el envío de campos vacíos y role="alert" permite anunciar el error a tecnologías de asistencia. La contraseña se oculta visualmente en el campo.

6. Casos de prueba previstos

Caso	Entrada	Resultado esperado
CP-01	Familiar: familia / admin	Acceso a bienvenida familiar
CP-02	Institucional, docente: docente / admin	Acceso a bienvenida docente
CP-03	Institucional, admin: admin / admin	Acceso a bienvenida administrativa
CP-04	Contraseña incorrecta	Mensaje de error
CP-05	Rol distinto al registrado	Mensaje de error
CP-06	Campos vacíos	Formulario impide el envío

Estos casos describen el comportamiento esperado de la implementación; no se adjuntan resultados automatizados ni se afirma haber ejecutado pruebas en un navegador.

7. Consideraciones técnicas y de seguridad

La codificación presentada constituye una demostración de front-end. Las credenciales permanecen en memoria y se comparan en el navegador; por ello, no debe utilizarse para gestionar usuarios reales ni información de menores. Una versión de producción requerirá API de autenticación, almacenamiento seguro de contraseñas, sesiones protegidas, validación del lado del servidor y autorización por rol e institución.

8. Conclusiones

Se desarrolló un módulo aislado de inicio de sesión coherente con el prototipo Pequeños Exploradores LMS. El componente emplea estados controlados, selección de modalidad, perfiles institucionales, validación de datos y presentación de errores. El archivo ZIP permite revisar el código del primer requerimiento sin incluir el resto del sistema.

9. Evidencia y anexos

Anexo A. Fragmento de código ejecutable: App.tsx, style.css, main.tsx y archivos de configuración, incluidos en el ZIP de entrega.

Anexo B. Código fuente del prototipo de referencia: la lógica original contiene chooseMode, login y la vista page === "auth" en src/App.tsx. La presente entrega es una adaptación aislada y no una copia íntegra del sistema.

Repositorio: puede incorporarse el PDF y el ZIP en la carpeta de la semana correspondiente.